

Commandes de base du système GNU/Linux

TME 0

Objectifs

- ★ Utiliser un terminal.
- ★ Oublier la souris.
- ★ Comprendre une arborescence.
- ★ Manipuler les fichiers.
- ★ Utiliser un gestionnaire de version.

Ce TP n'est pas noté mais est à rendre.

Rendu

- ⇒ Créer un répertoire rendu `TP0_numetu` où `votrenom` est votre nom et `numetu` est votre numéro étudiant..
- ⇒ Copier l'ensemble des fichiers à rendre dans ce répertoire.
- ⇒ Archiver le répertoire à l'aide de la commande suivante

```
tar czvf tp0_numetu.tgz TP0_numetu
```
- ⇒ Charger votre fichier sur Moodle dans la devoir TP0

Les fichiers à rendre vont nous servir à savoir si vous avez compris, n'hésitez pas à y ajouter des explications.

1 Introduction

1.1 GNU/Linux, pour quoi faire ?

Si vous étiez un bricoleur dans l'âme, doté d'une grande curiosité pour la technique, feriez-vous l'acquisition d'une automobile dont le constructeur aurait volontairement soudé le capot (vous interdisant ainsi l'accès au moteur), tout en vous contraignant à utiliser pour son entretien ou son fonctionnement quotidien des produits non-standards ? Assurément non ! Et pourtant, hier comme aujourd'hui, c'est presque ainsi que la plupart des ordinateurs vous sont vendus dans le commerce.

Bien qu'il serait très intéressant de comprendre le fonctionnement des composants matériels logés dans votre ordinateur, cela serait insuffisant pour rendre compte de toutes ses activités. De toute évidence, c'est au niveau du logiciel qu'il faut se placer. Que ce soit pour assouvir votre curiosité comme pour exprimer votre créativité, ou bien qu'il s'agisse d'adapter des applications existantes à vos besoins personnels, ou encore corriger des programmes défectueux, disposer des programmes tels qu'ils ont été écrits par les ingénieurs devient une nécessité. Sans eux, vous êtes condamnés à dépendre de la bonne volonté des entreprises éditrices des programmes que vous avez pourtant dûment achetés.

Depuis plus de 50 ans, pour contrer les logiques de propriétés qui instaurent cette dépendance technologique, et arguant du principe éthique que la connaissance est un bien commun qu'il faut pouvoir partager et enrichir librement, des informaticien·nes de toute nationalité ont, sur leur temps libre ou dans le cadre de leur activité professionnelle, conjugué leurs efforts pour bâtir une famille de systèmes d'exploitation, la plupart du temps gratuits, inspirée par toute ou partie de cette philosophie. Aujourd'hui, nombreux sont les téléphones intelligents, à être équipés des systèmes d'exploitations fiables et bien moins verrouillés que Microsoft Windows. Dans cette aventure, les universités ont joué un rôle non négligeable, et c'est aussi notre mission que de promouvoir et vous enseigner la science informatique pour ce qu'elle est, en vous affranchissant, autant que faire se peut, de toute influence étrangère à ses propres fins et avec l'une des toutes meilleures gammes d'outils qui soient : GNU/Linux.

Les ordinateurs des salles de TME sont sous *Debian 12 bookworm*. Debian est une distribution **GNU/Linux**. Il existe de nombreuses distributions, vous pouvez voir les distributions GNU/Linux au cours du temps (vous pouvez y retrouver Debian, ainsi que Android par exemple).

Les Raspberry-Pi que nous utiliserons pendant les projets sont aussi sous GNU/Linux, un OS qui est basé sur Debian (Raspberry PI OS anciennement Raspbian).

1.2 Prise en main / Geek style

Les environnements GNU/Linux sont en apparence semblables aux environnements classiques que vous connaissez (i.e. WINDOWS , MAC-OS, ...). Toutefois, les systèmes d'exploitation GNU/Linux sont historiquement réputés pour la souplesse qu'ils offrent dans la gestion des ressources (programmes, périphériques ...). Sous ces systèmes d'exploitation, le terminal est un outil privilégié qui permet aux utilisateurs de réaliser par le biais de commandes écrites de simples tâches que vous avez l'habitude de faire avec la souris (e.g. déplacer un fichier) ou des tâches bien plus sophistiquées et tout-à-fait hors de portée de quelques clics de souris. Cependant, la maîtrise du terminal s'apprend lentement ... mais sûrement !

1. Explorer le menu applications pour ouvrir un terminal.
2. Qu'est-il écrit ?
3. Taper le nom du navigateur internet suivi d'un & et appuyer sur la touche entrée. Tous les programmes avec une interface graphique évoluée peuvent être ainsi lancés grâce au terminal. L'exécution d'une commande est provoquée par l'utilisateur lorsqu'il presse la touche entrée. Vous veillerez à respecter les espacements au sein des commandes proposées par la suite.
4. A l'aide de votre souris et du menu des applications, créez un raccourci en faisant glisser l'icône du terminal sur le tableau de bord. Cela vous permettra d'ouvrir rapidement un terminal lors de vos sessions de travail ultérieures (ce procédé vaut également pour toute autre application que vous souhaiteriez pouvoir lancer rapidement sans explorer le menu des applications).
5. Relancer un terminal en cliquant sur l'icône que vous venez de glisser sur le tableau de bord. Lancer un navigateur suivi d'un &.
6. Tout en maintenant enfoncée la touche ALT avec votre pouce, presser la touche TAB (ou éventuellement ESC). Que se passe-t-il ?
7. A présent, maintenez enfoncées simultanément les touches CTRL et ALT (ou éventuellement SHIFT et ALT), et appuyez une fois sur la touche flèche droite. Que se passe-t-il ?

Exercice 1 Une promenade

1. Ouvrir un terminal.
2. Pour afficher le nom du répertoire courant, taper la commande **pwd** (pour s'en souvenir : print working directory). Lancer cette commande et noter le chemin absolu retourné par l'ordinateur.
3. La commande **cd ..** (**cd** pour change directory) permet de remonter au répertoire parent. La commande **ls** (pour list) permet de lister le contenu du répertoire courant. Effectuer autant de remontées vers le répertoire parent que vous le pourrez, tout en listant le contenu de chaque répertoire courant lors de chaque remontée. Vous afficherez aussi à chaque fois le nom du répertoire courant et vous observerez l'apparence de l'invite de commande de votre terminal.
4. Effectuer la descente inverse à présent en mentionnant à chaque fois le nom d'un sous-répertoire du répertoire courant vers lequel vous souhaitez vous déplacer (par exemple, si **bin** est un sous-répertoire du répertoire courant, taper **cd bin** pour vous déplacer vers ce sous-répertoire).
5. Effectuer la commande **cd /** puis **pwd**.
6. Taper **cd** pour revenir au répertoire racine de votre compte (vous pouvez également taper **cd ~**, le tilde désignant votre répertoire personnel).

Commandes utilisées

```
cd, pwd, ls
cd ..
cd /
cd ~
cd nom_de_repertoire
```

2 RTFM

Exercice 2 Utilisation du manuel

La suite des commandes utilisées devra être écrite dans un fichier. Vous pouvez écrire et éditer un fichier à l'aide d'un éditeur de texte (c'est le moment de commencer à en utiliser un).

Nous allons utiliser la commande **man** qui permet d'ouvrir le manuel d'utilisation de chaque commande.

1. Comment trouve-t-on les pages de manuel sur une commande donnée ? (taper `q` pour quitter).
2. Comment lit-on le manuel de la commande `man` ? Prenez le temps de lire la description.
3. Comment trouve-t-on l'ensemble des pages qui contiennent un mot donné ? C'est à dire si on ne connaît pas le nom de la commande et que l'on cherche un mot clé par exemple.
4. Comment demande-t-on la page de manuel de `signal` qui se trouve dans la section 7 ?
5. Comment visualiser toutes les sections du manuel relatives à une commande ?

Commandes utilisées

```
man
man nom_de_commande
man numero_section nom_de_commande
```

3 Le terminal

Répondre dans un fichier **terminal.txt**

Exercice 3 Ligne de commande du bash

La ligne de commande comporte de nombreuses commandes d'édition. Vous pourrez-vous rapporter à la section `READLINE` du manuel de `bash`. Ces commandes d'édition facilitent l'utilisation du terminal. Par exemple les flèches horizontales permettent de se déplacer d'un caractère sur la ligne courante.

1. Comment se déplacer d'un mot sur la ligne de commande ?
2. Comment échanger 2 mots de place ?
3. Pour copier/coller une commande de ou vers le terminal il faut utiliser les commandes `Maj-C-C` et `Maj-C-V`.
4. La complétion de nom (touche `TAB`) évite d'avoir à taper l'ensemble de noms des répertoires ou commandes si ceux-ci existent.
 - Retourner à la racine de votre répertoire ;
 - Taper `cd` B suivi de la touche `TAB`.
 - Que se passe-t-il si vous tapez `cd` M suivi de la touche `TAB`.
 - Même question lorsque vous tapez `e` suivi de la touche `TAB`.
5. Que font les flèches verticales ?
6. Que fait la commande **history** ?

Commandes utilisées

```
history
cd
```

4 Arborescence et manipulation de fichiers

Répondre dans un fichier *arbo.txt*

Exercice 4 Organiser vos informations sur votre compte personnel

A moins de considérer votre compte personnel comme un *cloud* en devenir, vous avez tout intérêt à organiser vos données au sein de votre répertoire personnel. L'objectif de cette section est de vous apprendre à créer et modifier une arborescence donnée ayant pour racine votre répertoire personnel en utilisant les commandes **mkdir**, **ls**, **cd**, **rm**, **rmdir**, **mv** et **touch**.

Vous pouvez toujours vous documenter avec la commande **man** sur l'ensemble des commandes.

1. Assurez-vous que vous êtes à la racine de votre compte.
2. Créer un répertoire `Info` et un répertoire `LeReste`.
3. Aller dans le répertoire `Info`.
4. Créer les répertoires `LU1IN021/TP1`, `LU1IN021/TP2` et `LU1IN021/Projet`.
5. Revenir à la racine du compte.
6. Créer les répertoires `Info/LU1IN021/TP3` et `Info/LU1IN021/TP4`.
7. Créer les répertoires `Projet/Info` et `Temp`.
8. Lister le contenu du répertoire courant.
9. Lister seulement les répertoires de `TP` de `Info/LU1IN021`.
10. Dessiner sur une feuille l'arborescence que vous venez de construire.
11. Trouver un logiciel de dessin et dessiner l'arborescence des répertoires que vous venez de construire. Enregistrer le fichier dans le répertoire `Info`.
12. Déplacer le fichier dans le répertoire `Info/LU1IN021/TP1`.
13. Renommer le fichier en `dessin_arbo` (avec la bonne extension).
14. Expliquer la différence entre la commande **cp** et **mv**.
15. Effacer le répertoire `Temp`.
16. Renommer le répertoire `Info` en `Informatique`.
17. Que fait la commande **touch** sur un fichier existant ? sur un fichier qui n'existe pas ? Créez un fichier dont la date de dernière modification est le 1^{er} janvier 2020 à 0h00.

Commandes utilisées

man, **cd**, **ls**, **mkdir**, **rm**, **rmdir**, **mv**

Pour aller plus loin vous pouvez jouer à Game Shell !

5 Gestionnaire de versions

Pourquoi un gestionnaire de version ?

En tant que informaticien·ne, qui jonglent avec un grand nombre de projets, fichiers répertoires . . . , on souhaite être capable facilement de

- Garder l'historique de version ;
- Essayer d'intégrer des nouvelles fonctionnalités sans casser l'existant ;
- Définir différentes versions d'un même logiciel ;
- Pouvoir revenir à des versions antérieures ;
- Travailler à plusieurs sur le même code.

Les logiciels de gestion de version sont fait pour ça !

Nous utiliserons **git**.

Git est un gestionnaire de version décentralisé le plus populaire actuellement. Git trouve ses origines dans le développement du noyau Linux et a été fondé par Linus Torvalds et publié sous licence libre GNU-GPLv2.. Il est aujourd'hui utilisé par un grand nombre de projet open source, comme par exemple Android, Eclipse ...

Un tutoriel est disponible ici [Tutoriel PPTI](#)

Exercice 5 Travailler seule avec Git (TP0/ex1)

1. Nous allons commencer par configurer Git en vous identifiant. Cela n'est à faire qu'une seule fois. **Si vous êtes en binôme, tout le monde doit faire les commandes ci-dessous !!**

```
git config --global user.name "votre_login"
git config --global user.email "votre_mail"
git config --global core.editor nano # ou votre éditeur préféré
git config --global color.ui auto
git config --global alias.lola 'log --graph --decorate --oneline --all'
git config --global alias.lolp 'log --pretty=format:"%h - %an, %ar : %s"'
```

2. Créer un répertoire de dépôt git, c'est à dire créer un répertoire normalement, puis s'y déplacer et enfin utiliser la commande

```
git init
```

qui va donc initialiser le répertoire courant comme étant maintenant sous le contrôle d'un gestionnaire de version.
Quels répertoires ont été créés ?

3. Créer un fichier dans ce répertoire.
4. Tapez la commande

```
git status
```

et comprendre le message.

5. Tapez la commande

```
git add votrefichier, quel est maintenant le statut de votre fichier ?
```

6. Vous pouvez maintenant commiter vos changements

```
git commit -m"Un superMessage"
```

Une fois le git commit exécuté vous obtenez un message récapitulatif portant sur le commit : vous trouverez d'une part le nom de la branche (branche du projet) ainsi que la somme de contrôle SHA 1 du commit.

7. Quel est maintenant le statut de votre répertoire ?
8. Expliquer maintenant le résultat de la commande

```
git log -p
```

9. Modifier maintenant votre fichier et comprendre le résultat de la commande

```
git diff
```

10. Vous allez maintenant lier votre dépôt local à un dépôt sur le serveur <https://gitlab.sorbonne-universite.fr>.
 - Créer un repository dans votre compte gitlab.su.
 - Dans votre répertoire appliquer la procédure décrite dans gitlab qui correspond à votre cas.
11. Que voyez-vous dans votre dépôt sur le serveur gitlab ?
12. Procédure pour mettre à jour les modifications sur le serveur :

- Modifier votre fichier.
- Ajouter le fichier pour mettre à jour ce qui sera commiter

```
git add filename
```

- Commiter votre fichier

```
git commit -m"Un message"
```

- Pusher les commits.

Il est possible de générer des mots de passes temporaires appelés *token* Pour cela, dans vos settings → Access Token , ajouter un nouveau token et cocher api. Enregistrer bien ce token il ne sera plus visible par la suite.

```
git push
```

Vérifier que vos commits sont bien arrivés sur le serveur.

Exercice 6 Travailler à plusieurs (TP0/ex2)

1. Aller dans un autre répertoire que la question précédente.
2. Clonez le répertoire <https://gitlab.su.sorbonne-universite.fr/ullin021/2026/tp0.git>

```
$ git clone https://gitlab.su.sorbonne-universite.fr/ullin021/2026/tp0.git
```

Quels répertoires ont été créés ?

3. Déplacez vous dans le répertoire versionné, que retourne la commande **git status**.
4. Créer un fichier `votre_nom.txt` où `votre_nom` est votre nom ou votre login.
5. Ajouter votre fichier sur le dépôt distant (voir la procédure question précédente).
6. Que se passe-t-il lors du push ? (si vous n'arrivez pas à pusher, envoyer un mail à cecile.braunstein@lip6.fr en indiquant votre login github).
7. Mettre à jour votre copie locale (le répertoire que vous avez cloner) avec la commande `git pull`
8. Réessayer de *pusher* (comme beaucoup d'élèves vont pusher en même temps il va falloir mettre à jour plusieurs fois votre copie locale avant de pouvoir pusher).
9. Modifier la première ligne de votre fichier et celle de votre voisin.
10. Commiter (*add/commit*) vos changements et envoyer les sur le serveur (*push*).
11. Que se passe-t-il ?
12. Régler le conflit sur votre fichier (choisissez la ou les modifications que vous voulez garder), commiter et pusher.
13. Regarder l'historique des changements avec la commande **git lola** et **git lop** ou l'interface graphique `gitg` ou sur le serveur.